CSA0604-Design and Analysis of Algorithms

Q25. MINIMUM SPANNING TREE FOR NETWORK DESIGN

Implementation of Kruskal's and Prim's Algorithms and Performance Comparison

Student Information	Details
Name	**K. Baba Satya Kumar**
Register Number	**192525226**
Course Faculty	**Dr M Manikandan**
Work Type	Individual
Programming Language	Python
Technique	Greedy, Kruskal, Prim
Course Outcomes	CO1, CO3

1. Problem Statement

A telecommunications company wants to connect all offices with minimum infrastructure cost. Implement Kruskal's and Prim's algorithms to construct the Minimum Spanning Tree and compare their performance.

The assignment specifies the following constraints:

- Number of vertices: $1 \leq V \leq 50{,}000$.
- Number of edges: $1 \leq E \leq 200{,}000$.
- All edge weights are positive.
- The graph is connected.
- Theoretical time complexity and actual execution time must be compared.

The problem statement, constraints, techniques and CO mapping above are taken directly from Q25 of the supplied assignment document.

2. Objectives

- To understand the concept and properties of a Minimum Spanning Tree (MST).
- To apply the greedy strategy to a weighted, undirected, connected graph.
- To implement Kruskal's algorithm using edge sorting and Disjoint Set Union (DSU).
- To implement Prim's algorithm using an adjacency list and a binary min-heap.
- To calculate the total infrastructure cost represented by the MST.
- To verify that both algorithms produce a minimum-cost spanning tree.
- To compare the algorithms using time complexity, space complexity, data structures and execution time.
- To discuss suitability and scalability for large network-design inputs.

3. Introduction to Minimum Spanning Tree

A graph can be used to represent a network in which vertices represent offices and weighted edges represent possible communication links. The edge weight can represent infrastructure cost, installation cost or another additive measure of deployment cost.

A Minimum Spanning Tree is a spanning tree of a connected, weighted, undirected graph having the minimum possible sum of edge weights. Because it is a tree, it connects every vertex without cycles and contains exactly $V - 1$ edges.

MST contains exactly $V - 1$ edges

MST Cost = Σ w(e), for every selected edge e

If multiple edges have the same weights, more than one different MST can exist. Even in that situation, every valid MST has the same minimum total weight.

4. Important MST Properties

- Connectivity: every vertex is reachable from every other vertex through the selected MST edges.
- Acyclic property: the selected edges do not contain a cycle.
- Edge count: a spanning tree with V vertices contains exactly $V - 1$ edges.
- Minimum-weight property: the sum of the selected edge weights is no greater than that of any other spanning tree.
- Cut property: for a cut of the graph, a minimum-weight crossing edge is safe for an MST; with ties, more than one safe choice may exist.
- Cycle property: for a cycle, an edge that is strictly heavier than another suitable edge in the cycle cannot be required in a minimum spanning tree.

5. Greedy Strategy Used

Both algorithms follow a greedy strategy. At every step, the algorithm selects a locally minimum-cost edge that is safe to add to the growing solution. The important difference is how the candidate edge is chosen.

Feature	Kruskal	Prim
Selection view	Global edge order	Current tree frontier
Starting point	No fixed start vertex	Starts from one selected vertex
Main structure	Sorted edge list + DSU	Adjacency list + min-heap
Cycle prevention	DSU component test	Visited-vertex test

6. Kruskal's Algorithm

Kruskal's algorithm processes all edges in non-decreasing order of weight. Initially, every vertex forms a separate connected component. An edge is accepted if its two endpoints belong to different components. After acceptance, the two components are merged. This prevents a cycle from being introduced.

1. Read the vertices and weighted edges.
2. Sort all edges in non-decreasing order of weight.
3. Create a separate DSU set for each vertex.
4. Examine the next smallest edge.
5. Use FIND to determine the component of each endpoint.
6. If the endpoints belong to different components, accept the edge and UNION the components.
7. If they are already in the same component, reject the edge because it would create a cycle.
8. Stop when $V - 1$ edges have been accepted.

7. Disjoint Set Union Used in Kruskal

The DSU structure maintains a collection of disjoint sets representing the connected components created during Kruskal's execution. Two optimizations are used: path compression in FIND and union by rank in UNION. These optimizations make a sequence of DSU operations extremely efficient in practice.

Operation	Purpose
MAKE-SET	Creates a separate set for each vertex.
FIND(x)	Returns the representative of the set containing x.
UNION(x,y)	Merges the two different sets containing x and y.
Cycle test	If FIND(x) = FIND(y), adding edge (x,y) would create a cycle.

8. Kruskal Pseudocode

```
KRUSKAL(V, E)
    sort E by non-decreasing weight
    make-set(v) for every vertex v
    MST = empty set
    total = 0

    for each edge (u, v, w) in E:
        if FIND(u) != FIND(v):
            add (u, v, w) to MST
            total = total + w
            UNION(u, v)

        if |MST| = V - 1:
            break

    return MST, total
```

9. Kruskal Correctness Argument

At each iteration, Kruskal considers the lightest remaining edge. If its endpoints are in different components, the edge connects two separate components without creating a cycle. By the MST cut property, such a minimum-weight safe edge can be included in an MST. Repeating this process until $V - 1$ edges are selected produces a spanning tree whose total weight is minimum.

10. Kruskal Complexity

Sorting all edges: O(E log E)

DSU operations: O(E α(V)) approximately

Overall time complexity: O(E log E)

Auxiliary graph/DSU storage: O(V + E)

Here α(V) is the inverse Ackermann function. It grows so slowly that DSU operations are effectively near-constant for practical input sizes. The sorting of the E edges is the dominant asymptotic operation in the implementation.

11. Prim's Algorithm

Prim's algorithm starts from a selected vertex and grows a single tree. At every stage, it chooses the minimum-weight edge that connects a vertex already in the tree to a vertex outside the tree. A binary min-heap is used to efficiently obtain the smallest available candidate.

9. Choose a starting vertex.
10. Mark the starting vertex as visited.
11. Insert its incident edges into the min-heap.
12. Remove the minimum-weight candidate edge.
13. If the destination vertex is already visited, discard that candidate.
14. Otherwise, accept the edge, add the new vertex to the MST and mark it visited.
15. Insert the newly available edges from that vertex into the heap.
16. Continue until V − 1 edges have been selected.

12. Prim Pseudocode

```
PRIM(V, Adj)
    choose start vertex s
    visited[s] = true
    insert edges from s into min-heap
    MST = empty set
    total = 0

    while heap is not empty and |MST| < V - 1:
        remove minimum edge (w, u, v)

        if visited[v]:
            continue

        add (u, v, w) to MST
        total = total + w
        visited[v] = true

        for each edge (v, x, weight) in Adj[v]:
            if not visited[x]:
                insert (weight, v, x) into heap

    return MST, total
```

13. Prim Correctness Argument

At every step, Prim chooses a minimum-weight edge crossing from the already constructed tree to an unvisited vertex. This edge is safe by the MST cut property. Therefore, each accepted edge can be part of an MST. After V − 1 safe edges are selected, all vertices are connected and the resulting tree is a minimum spanning tree.

14. Prim Complexity

Adjacency list + binary min-heap: O(E log V)

Space complexity: O(V + E)

The adjacency list stores both directions of each undirected edge. Heap insertion and removal require logarithmic time. This implementation is appropriate for sparse or moderately sparse graphs because it stores only the edges that actually exist.

15. Worked Example

Consider five offices represented by vertices 1 to 5. The following weighted edges represent possible communication links.

Possible Link	Infrastructure Cost
1–2	2
1–3	1
2–3	5
2–4	10
3–4	3
3–5	7
4–5	1
2–5	6

16. Kruskal Trace for the Worked Example

Order	Edge	Weight	Decision	Explanation
1	1–3	1	Accept	Connects two different components.
2	4–5	1	Accept	Connects two different components.
3	1–2	2	Accept	Connects vertex 2 to the current component.
4	3–4	3	Accept	Connects the two remaining components.
5	2–5	6	Reject	Endpoints are already connected; would form a cycle.

MST Cost = 1 + 1 + 2 + 3 = 7

Number of MST edges = 5 − 1 = 4

17. Prim Trace for the Worked Example

Step	Selected Edge	Weight	New Vertex	Reason
1	1–3	1	3	Minimum edge from start vertex 1.
2	3–4	3	4	Cheapest edge from the current tree to an unvisited vertex.
3	4–5	1	5	Cheapest edge from the current tree to vertex 5.
4	1–2	2	2	Connects the last unvisited vertex.

MST Cost = 1 + 3 + 1 + 2 = 7

The edge order differs from Kruskal because Prim grows the tree from a starting vertex. Nevertheless, both produce a valid MST with the same minimum total cost.

18. Comparison of Kruskal and Prim

Parameter	Kruskal	Prim
Design technique	Greedy	Greedy
Selection	Globally lightest remaining edge	Lightest edge crossing current tree cut
Data structure	DSU + sorted edge list	Adjacency list + binary min-heap
Time complexity	O(E log E)	O(E log V)
Space complexity	O(V + E)	O(V + E)
Cycle handling	Explicit with DSU	Implicit through visited set
Start vertex	Not required	Required
Graph representation	Edge list is natural	Adjacency list is natural
Disconnected input	Can produce a minimum spanning forest	`Single run from one start covers one component; multiple starts are needed for a forest`
Best practical fit	Convenient when edges are already available as a list	Convenient when adjacency traversal is natural

19. Python Implementation

The following implementation follows the assignment requirement and uses only standard Python modules. Kruskal uses DSU with path compression and union by rank. Prim uses an adjacency list and heapq. The program also measures execution time using time.perf_counter().

```
import heapq
import time

class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        a = self.find(a)
        b = self.find(b)
        if a == b:
            return False
        if self.rank[a] < self.rank[b]:
            a, b = b, a
        self.parent[b] = a
        if self.rank[a] == self.rank[b]:
            self.rank[a] += 1
        return True
```

```
def kruskal(vertices, edges):
    dsu = DSU(vertices)
    mst = []
    total = 0

    for w, u, v in sorted(edges):
        if dsu.union(u, v):
            mst.append((u, v, w))
            total += w
            if len(mst) == vertices - 1:
                break

    return mst, total
```

```
def prim(vertices, edges):
    adj = [[] for _ in range(vertices)]

    for w, u, v in edges:
        adj[u].append((w, v))
        adj[v].append((w, u))

    visited = [False] * vertices
    heap = [(0, 0, -1)]
    mst = []
    total = 0

    while heap and len(mst) < vertices - 1:
        w, v, parent = heapq.heappop(heap)

        if visited[v]:
            continue

        visited[v] = True

        if parent != -1:
            mst.append((parent, v, w))
            total += w

        for nw, nv in adj[v]:
            if not visited[nv]:
                heapq.heappush(heap, (nw, nv, v))

    return mst, total
```

```
v = int(input("Enter number of vertices: "))
e = int(input("Enter number of edges: "))
edges = []

for _ in range(e):
    u, w, cost = map(int, input().split())
    edges.append((cost, u - 1, w - 1))

start = time.perf_counter()
kmst, kcost = kruskal(v, edges)
kend = time.perf_counter()

start2 = time.perf_counter()
pmst, pcost = prim(v, edges)
end2 = time.perf_counter()

print("\nKruskal MST:")
for u, w, cost in kmst:
    print(u + 1, w + 1, cost)
print("Total cost:", kcost)
print("Execution time:", kend - start, "seconds")

print("\nPrim MST:")
for u, w, cost in pmst:
```

```
    print(u + 1, w + 1, cost)
print("Total cost:", pcost)
print("Execution time:", end2 - start2, "seconds")
print("Minimum costs equal:", kcost == pcost)
```

20. Explanation of the Implementation

Part	Purpose
DSU.find()	Finds the representative of a connected component using path compression.
DSU.union()	Merges two different components using union by rank.
sorted(edges)	Produces non-decreasing edge order for Kruskal.
adj	Stores the undirected graph as an adjacency list for Prim.
heapq	Maintains a min-heap so the smallest candidate edge can be removed efficiently.
visited	Prevents Prim from adding an already included vertex.
perf_counter()	Measures elapsed execution time with a high-resolution performance counter.

21. Sample Input

```
5
8
1 2 2
1 3 1
2 3 5
2 4 10
3 4 3
3 5 7
4 5 1
2 5 6
```

22. Expected MST Result

Algorithm	Selected MST Edges	Total Cost	Edge Count
Kruskal	1–3, 4–5, 1–2, 3–4	7	4
Prim	1–3, 3–4, 4–5, 1–2	7	4

The order of edges printed by the program may differ because Prim depends on its starting vertex and heap ordering. The important correctness checks are that the graph is connected, the MST has $V - 1$ edges, and both algorithms obtain the same minimum total cost for this test case.

23. Execution-Time Comparison

Execution time is machine-dependent. It should not be invented or treated as a universal value. Run the supplied program on the same computer and record the displayed values.

Input Size	Vertices (V)	Edges (E)	Kruskal Time (s)	Prim Time (s)
Test 1	5	8	Record after execution	Record after execution
Test 2	100	500	Record after execution	Record after execution
Test 3	1,000	5,000	Record after execution	Record after execution
Test 4	5,000	20,000	Record after execution	Record after execution
Test 5	10,000	50,000	Record after execution	Record after execution

For a fair comparison, use the same input graph, the same Python version and the same computer for both algorithms. For very small graphs, timing differences can be dominated by interpreter and measurement overhead, so larger inputs are more informative.

24. Performance Analysis

Kruskal performs a global sorting operation on all E edges. As E increases, the sorting stage becomes a major component of execution time. Its DSU operations are highly efficient after path compression and union by rank.

Prim processes graph adjacency information and repeatedly extracts the minimum candidate edge from a binary heap. With an adjacency list and binary heap, the implementation has O(E log V) time complexity and O(V + E) space complexity.

The actual faster algorithm for a particular input cannot be concluded from asymptotic notation alone. Hardware, Python implementation overhead, graph density, input ordering and the exact implementation can affect measured execution time.

25. Test Cases and Verification

Test Case	Graph Condition	Expected Check
1	Basic connected graph	Both algorithms return V − 1 edges and equal cost.
2	Already a tree	All V − 1 edges are selected.
3	Several equal-weight edges	Different valid MST edge sets may occur, but minimum cost should agree.
4	Dense connected graph	Both algorithms should remain correct; execution time can be compared.
5	Large graph within assignment limits	Check correctness first, then compare measured execution time.

26. Correctness Verification Checklist

- The input graph must be connected for a single MST to exist.
- The returned MST must contain exactly V − 1 edges.
- The selected edges must connect every vertex.
- No cycle may be present in the final selected edges.
- Kruskal must accept an edge only when its DSU representatives are different.
- Prim must accept an edge only when its destination vertex is unvisited.
- The total cost is the sum of the weights of the selected MST edges.
- For the same connected weighted graph, Kruskal and Prim must return the same minimum total weight, although their selected edge sets can differ when multiple MSTs exist.

27. Space Complexity Analysis

Component	Kruskal	Prim
Input edge storage	O(E)	O(E) through adjacency list
Vertex-related storage	O(V) for DSU	O(V) for visited and heap-related structures
MST storage	O(V)	O(V)
Total asymptotic space	O(V + E)	O(V + E)

28. Scalability for the Given Constraints

The assignment permits up to 50,000 vertices and 200,000 edges. A representation that explicitly stores a $V \times V$ matrix would require $\Theta(V^2)$ entries, which is unnecessarily large for many sparse network graphs. The chosen implementations use edge lists and adjacency lists, requiring $O(V + E)$ storage.

For Kruskal, the main scalability consideration is sorting up to 200,000 edges. For Prim, the adjacency list and binary heap avoid the $O(V^2)$ behavior of a simple adjacency-matrix implementation. Therefore, both selected implementations are appropriate choices for the stated constraints, subject to available system memory and execution environment.

29. Practical Trade-Offs

Situation	Preferred Approach	Reason
Edge list is readily available	Kruskal	Sorting the edge list and using DSU is straightforward.
Adjacency list is readily available	Prim	The algorithm naturally explores adjacent edges.
Need explicit cycle detection	Kruskal	DSU directly represents connected components.
Need to grow one connected network from a chosen office	Prim	Starts from the chosen vertex and expands the tree.
Very sparse graph	Either	Both implementations have near-linear graph-storage requirements; input representation matters.

30. Applications

- Telecommunications network design.
- Computer network and cable planning.
- Electrical power distribution planning.
- Road and transportation infrastructure planning.
- Pipeline and utility network design.
- Clustering and graph-based data analysis in suitable formulations.

31. Advantages and Limitations

Algorithm	Advantages	Limitations
Kruskal	`Simple greedy logic; natural for edge lists; DSU provides efficient cycle detection; works well when edge sorting is convenient.`	Requires sorting the complete edge set; performance is influenced by the number of edges.

Prim	Naturally grows a connected tree; adjacency-list and heap implementation is effective for large sparse graphs; no explicit global edge sort is required.	Requires a graph adjacency structure; heap operations add implementation overhead.

32. Result

Kruskal's and Prim's algorithms were implemented for the Minimum Spanning Tree network-design problem. For the worked five-vertex graph, both algorithms selected four edges and produced a minimum total infrastructure cost of 7. Kruskal used an edge-sorting and DSU approach, while Prim used an adjacency list and a binary min-heap. The theoretical complexities of the implemented versions are O(E log E) for Kruskal and O(E log V) for Prim, with O(V + E) space for both.

33. Discussion

The experiment demonstrates that different greedy decision processes can solve the same MST optimization problem. Kruskal views the graph globally through its ordered edge list, while Prim maintains a growing connected tree and selects the cheapest edge crossing its current boundary. The equality of the minimum cost for the worked example confirms the correctness of the implementations for that input.

For performance evaluation, theoretical complexity gives the expected growth trend, while execution-time measurement gives evidence for the particular machine and test data. A high-quality experimental comparison should therefore report both the theoretical complexity and measured timings for multiple input sizes.

34. Conclusion

Minimum Spanning Tree algorithms provide an effective mathematical model for connecting all offices in a network while minimizing total infrastructure cost. In this work, Kruskal's and Prim's greedy algorithms were implemented and analyzed using Python. Kruskal's algorithm sorts the candidate edges and uses Disjoint Set Union to accept only edges that connect different components. Prim's algorithm begins from a selected vertex and repeatedly chooses the least-cost edge that expands the current tree to an unvisited vertex.

For the worked graph, both methods generated a valid MST containing $V - 1 = 4$ edges and achieved the same minimum cost of 7. This illustrates an important property of MST algorithms: the edge-selection order can differ while the final minimum total weight remains the same. The implementations use efficient graph representations suitable for the assignment limits of 50,000 vertices and 200,000 edges.

From the complexity analysis, Kruskal's implementation requires O(E log E) time because of edge sorting, while Prim's adjacency-list and binary-heap implementation requires O(E log V) time. Both require O(V + E) space. Therefore, the algorithm should be selected according to graph representation, input characteristics and implementation requirements rather than relying only on one complexity expression. The measured execution-time comparison should be obtained by running the program on the same hardware and input datasets.

35. References

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022. Chapter 21, "Minimum Spanning Trees," begins on p. 585. ISBN: 9780262046305.

2. R. Sedgewick and K. Wayne, Algorithms, 4th ed., Addison-Wesley Professional, 2011. Section 4.3, "Minimum Spanning Trees."

3. Princeton University, Algorithms 4th Edition, Section 4.3, "Minimum Spanning Trees." https://algs4.cs.princeton.edu/43mst/

4. Python Software Foundation, Python 3.14 Documentation, "heapq — Heap queue algorithm." https://docs.python.org/3/library/heapq.html

5. Python Software Foundation, Python 3.14 Documentation, "time — Time access and conversions." https://docs.python.org/3/library/time.html

36. Complete Python Implementation

The complete program below implements Kruskal's and Prim's algorithms using standard Python data structures. The program accepts a connected weighted undirected graph, computes both MSTs, displays the selected edges and total cost, and measures execution time. The source code is intentionally presented without comments.

Welcome × exp4.py × imple.py ●

imple.py > ...

```python
import heapq
import time

class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        a = self.find(a)
        b = self.find(b)
        if a == b:
            return False
        if self.rank[a] < self.rank[b]:
            a, b = b, a
        self.parent[b] = a
        if self.rank[a] == self.rank[b]:
            self.rank[a] += 1
        return True

def kruskal(vertices, edges):
    dsu = DSU(vertices)
    mst = []
    total = 0

    for w, u, v in sorted(edges):
        if dsu.union(u, v):
            mst.append((u, v, w))
            total += w
            if len(mst) == vertices - 1:
                break

    return mst, total

def prim(vertices, edges):
    adj = [[] for _ in range(vertices)]

    for w, u, v in edges:
        adj[u].append((w, v))
        adj[v].append((w, u))

    visited = [False] * vertices
    heap = [(0, 0, -1)]
    mst = []
    total = 0

    while heap and len(mst) < vertices - 1:
        w, v, parent = heapq.heappop(heap)
```

```
55
56          if visited[v]:
57              continue
58
59          visited[v] = True
60
61          if parent != -1:
62              mst.append((parent, v, w))
63              total += w
64
65          for nw, nv in adj[v]:
66              if not visited[nv]:
67                  heapq.heappush(heap, (nw, nv, v))
68
69      return mst, total
70
71  v = int(input("Enter number of vertices: "))
72  e = int(input("Enter number of edges: "))
73  edges = []
74
75  for _ in range(e):
76      u, w, cost = map(int, input().split())
77      edges.append((cost, u - 1, w - 1))
78
79  start = time.perf_counter()
80  kmst, kcost = kruskal(v, edges)
81  kend = time.perf_counter()
82
83  start2 = time.perf_counter()
84  pmst, pcost = prim(v, edges)
85  end2 = time.perf_counter()
86
87  print("\nKruskal MST:")
88  for u, w, cost in kmst:
89      print(u + 1, w + 1, cost)
90  print("Total cost:", kcost)
91  print("Execution time:", kend - start, "seconds")
92
93  print("\nPrim MST:")
94  for u, w, cost in pmst:
95      print(u + 1, w + 1, cost)
96  print("Total cost:", pcost)
97  print("Execution time:", end2 - start2, "seconds")
98  print("Minimum costs equal:", kcost == pcost)
```

37. Sample Input and Output

```
PS C:\Users\newadmin\Downloads\exps> python imple.py
Enter number of vertices: 5
Enter number of edges: 8
1 2 2
1 3 1
2 3 5
2 4 10
3 4 3
3 5 7
4 5 1
2 5 6

Kruskal MST:
1 3 1
4 5 1
1 2 2
3 4 3
Total cost: 7
Execution time: 6.67000003886642e-05 seconds

Prim MST:
1 3 1
1 2 2
3 4 3
4 5 1
Total cost: 7
Execution time: 2.4900000425986946e-05 seconds
Minimum costs equal: True
```

The sample graph has five vertices, so a valid MST must contain four edges. Both algorithms produce total cost 7. The execution-time values must be obtained by running the program on the target system.